Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Tek yönlü bağlı listeden (singly linked list) eleman silme
- Silme işleminde dikkat edilecek durumlar: boş liste, ilk elemanın silinmesi, elemanın listede olmaması
- Dairesel tek yönlü bağlı liste (circular singly linked list) ve `last` göstericisi
- Dairesel listede başa ekleme (`insertAtFront`) ve sona ekleme (`insertAtLast`)
- Dairesel listeyi yazdırma (`printList`)
- Çift yönlü bağlı liste (doubly linked list): `prev` ve `next` göstericileri
- Çift yönlü listede başa ekleme
- Çift yönlü listenin avantajı ve dezavantajı

Hocanın Özellikle Vurguladığı Kısımlar

- Bağlı liste mantığı yığın, kuyruk, ağaç ve graf gibi veri yapılarının tasarımında kullanılır. Bu yüzden ekleme ve silme iyi öğrenilmelidir. İleride bu işlemler yeniden anlatılmayacak.
- Algoritma tasarlarken önce uç noktaları ve olası durumları belirleyin. Sonra algoritmayı bu durumları karşılayacak şekilde kurun. Böylece yama yapılmış bir algoritma ortaya çıkmaz.
- Silinecek elemanın bir önceki elemanının adresi saklanmalıdır. Aksi hâlde önceki eleman, silinen elemandan sonrakine bağlanamaz.
- Silinen düğüm `free` ile serbest bırakılmalıdır. Bırakılmazsa çalışma sırasında bellek dolar ve yeni yer ayrılamaz.
- Döngü koşulunda önce `current != NULL` kontrol edilmelidir. `current` NULL ise `current->value` diye bir şey yoktur.
- Döngüden çıkınca da önce `current == NULL` kontrol edilir. Böylece neden çıkıldığı güvenle anlaşılır.
- Aradaki bir elemanı silmekle son elemanı silmek arasında fark yoktur. Tek farklı durum ilk elemanın silinmesidir, çünkü `head` değişmelidir.
- Dairesel listede ilk eleman `last->next`, son eleman `last`'tır.
- Dairesel listede başa ekleme ile sona ekleme arasındaki tek fark: sona eklerken `last = newNode` yapılır.
- Çift yönlü liste, ancak algoritma her elemanın öncekini sürekli bilmeye ihtiyaç duyuyorsa seçilmelidir. Aksi hâlde her düğümdeki ek gösterici boşuna yer kaplar.
- `create` fonksiyonunda `newNode->prev = NULL` yapılırsa ekleme fonksiyonlarında bunu ayrıca yazmaya gerek kalmaz.

Kısa Tekrar Notları

- Silme işleminin durumları (hoca üç durum sayar: liste boş, eleman var, eleman yok; ilk elemanın silinmesi sonradan ayrı durum olarak eklenir):
  - Liste boş (`head == NULL`): "Empty List" yaz, işlem yapma.
  - Silinecek eleman ilk eleman: `tmp = head; head = head->next; free(tmp);`
  - Silinecek eleman arada ya da sonda: `prev->next = current->next; free(current);`
  - Eleman listede yok: döngü `current == NULL` olunca biter, "not exist" yazılır.
- Tek yönlü listede her düğümün tek bir göstericisi (`next`) vardır.

- Dairesel listede son düğüm ilk düğümü gösterir. Erişim için `last` adında bir gösterici tutulur.
- Dairesel listeye tek eleman eklenince düğüm kendini gösterir: `newNode->next = last`.
- Çift yönlü listede her düğümde `prev` ve `next` vardır. İlk düğümün `prev`'i ve son düğümün `next`'i NULL'dur.
- Çift yönlü listenin avantajı: önceki elemanın adresini geçici bir değişkende tutmak gerekmez.
- Çift yönlü listenin dezavantajı: her düğüm için fazladan bir gösterici, yani yer kaybı.
- Bellekte ayrılan adreslerin sıralı olması gerekmez. Örneklerdeki 1000, 1200, 1500 gibi adresler gelişigüzel seçilmiştir. Gerçek adresler `printf("%p", p)` ile görülebilir.

Detaylı Açıklamalar

Bağlı Listeden Eleman Silme

Örnek liste: `head` 1000'i gösterir. 1000'de 7 vardır ve 1500'ü gösterir. 1500'de 9 vardır ve 1700'ü gösterir. 1700'de 12 vardır ve NULL'u gösterir.

- 9 silinmek istenirse önce 9 bulunur. Sonra 7'nin artık 9'u değil, 9'un gösterdiği 12'yi göstermesi gerekir.
- Bunun için 9'a nereden gelindiğinin (önceki elemanın) bilinmesi gerekir.
- 1500 numaralı düğüm `free` ile serbest bırakılır. C'de ayrılan yer heap bölgesinden alınır. Serbest bırakılmazsa başka ayırmalar için kullanılamaz.

Silme algoritmasının akış diyagramı (ekrandan, 00:29:30):

```
oku: value
head == NULL ?
  T: yaz "Empty List"
  F: head->value == value ?
       T: tmp = head
          head = head->next
          free(tmp)
       F: prev = head
          current = head->next
          (current != NULL) && (current->value != value) ?
            T: prev = current
               current = current->next
               -> koşula geri dön
            F: current == NULL ?
                 T: yaz "not exist"
                 F: prev->next = current->next
                    free(current)
```

- Hoca akışı C koduna yakın yazdı. Öğrencilerden bunu bir `delete` fonksiyonuna kendilerinin çevirmesini istedi.
- `head` global bir değişkendir: `NODE *head;`

İlk Elemanın Silinmesi

- Örnek: `head` 1000'i gösterir. 1000'de 5 vardır ve 1200'ü gösterir.
- `head->value`, `head`'in gösterdiği adresteki (1000) düğümün değeridir. `head` yalnızca bir göstericidir, kendi değeri yoktur.
- 5 silinecekse:
  - `tmp = head` (1000 saklanır, çünkü serbest bırakılacak)
  - `head = head->next` (`head` artık 1200'ü gösterir)
  - `free(tmp)` (1000 serbest bırakılır)

Aradaki veya Sondaki Elemanın Silinmesi Örnek liste: 1000'de 5, 1200'de 7, 1500'de 9. Silinecek değer 9 (ekrandan, 00:29:30):

Adım	prev	current	current->value
Başlangıç	1000	1200	7
$7 \neq 9$, ilerle	1200	1500	9
$9 = 9$, döngüden çık	1200	1500	9

- Döngüden çıkınca `current` NULL değildir. Demek ki eleman bulunmuştur.
- `prev->next = current->next` ile 1200'deki düğüm artık NULL'u gösterir.
- `free(current)` ile 1500 serbest bırakılır.
- 7 silinseydi de aynı işlem yapılırdı: 1000'deki düğüm 1500'ü gösterirdi.

Dairesel Tek Yönlü Bağlı Liste (Circular Singly Linked List)

- Bazı durumlarda son elemanın ilk elemanı göstermesi gerekir. Örneğin kuyruklarda bu yapı avantajlıdır. Kuyruklar sonraki hafta anlatılacak.
- Listeye erişmek için yine bir başlangıç noktası gerekir. Normal listede bu `head` idi.
- Dairesel listede son elemanın adresi `last` göstericisinde tutulursa ilk elemana da erişilir:
  - İlk eleman: `last->next`
  - Son eleman: `last`
- Başlangıçta `last` NULL'dur: `NODE *last = NULL;`
- Ekleme ihtiyaca göre başa, sona veya araya yapılabilir. Örneğin liste bir kuyruğu temsil ediyorsa yeni gelen sona eklenir. Sıralı bir listede ise yeni eleman sırayı bozmayacak yere eklenir.
- Hoca `create` fonksiyonunu `insert`'ün içinde çağırmadı. Böylece başka bir yerde oluşturulmuş bir düğüm de eklenebilir.

Başa ve sona ekleme (ekrandan, 01:11:00):

```
NODE *last = NULL;

void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    if (last == NULL)            /* bağlı liste boş */
    {
        last = newNode;
        newNode->next = last;    /* tek düğüm kendini gösterir */
    }
    else
    {
        newNode->next = last->next;   /* yeni düğüm eski ilk elemanı gösterir */
        last->next = newNode;         /* son eleman yeni ilk elemanı gösterir */
    }
}

void insertAtLast(NODE *newNode, int value)
{
    newNode->value = value;
    if (last == NULL)
    {
        last = newNode;
        newNode->next = last;
    }
    else
    {
        newNode->next = last->next;
        last->next = newNode;
        last = newNode;               /* tek fark: yeni düğüm artık son eleman */
    }
}
```

- `newNode->value = value` ataması liste boş da olsa dolu da olsa yapıldığı için `if` dışına alındı.
- Bir öğrenci boş liste durumunda `last = newNode` satırının alta alınmasını önerdi. Hoca kabul etmedi: o zaman `newNode->next` atanırken `last` hâlâ NULL olurdu ve düğüm kendini göstermezdi.
- Hoca önce fonksiyonu `return last;` ile yazdı. Bir öğrenci `last`'ın global olduğunu hatırlatınca dönüş tipi `void` yapıldı. Değişken fonksiyonlar arasında parametre olarak aktarılacaksa döndürmek gerekir.

Başa ekleme örneği:

Durum	Liste (ilk elemandan başlayarak)	last
Boş listeye 8 (1000)	8 → 8	1000
Başa 4 (1200)	4 → 8 → 4	1000
Başa 7 (800)	7 → 4 → 8 → 7	1000

- Başa eklemek, `last`'ın arkasına eklemek demektir. `last` değişmez.
- 7 eklenince önce `newNode->next = last->next` ile 7, 4'ü gösterir. Sonra `last->next = newNode` ile 8, 7'yi gösterir.

Sona ekleme örneği: liste 5 (1000) → 9 (1500) ve `last` = 1500. 8 (1800) sona eklenir:

- `newNode->next = last->next`: 8, 5'i gösterir.
- `last->next = newNode`: 9, 8'i gösterir.
- `last = newNode`: `last` 1800 olur.
- Sonuç: 5 → 9 → 8 → 5.

Dairesel Listeyi Yazdırma

Hocanın tahtaya yazdığı kod (ekrandan, 01:17:20):

```
void printList()
{
    NODE *p;
    if (last == NULL)
        printf("List is empty\n");
    else
        for (p = last->next; p != last->next; p = p->next)
            printf("%d\n", p->value);
}
```

- Yazdırmaya ilk elemandan, yani `last->next`'ten başlanır. Tekrar ilk elemana gelince durulur.
- Düğümün bellekteki adresini görmek için `printf("%p %d\n", p, p->value);` yazılabilir.
- Bu koddaki döngü koşulu hatalıdır. Ayrıntı ve düzeltme "Ders sonrası notlar" bölümündedir.

Çift Yönlü Bağlı Liste (Doubly Linked List)

- Tek yönlü listede her eleman yalnızca bir elemanı, yani kendinden sonrakini gösterir.
- Araya ekleme ya da silme için ise önceki elemanın yeri bilinmelidir. Örneğin 2 → 5 listesine sırayı bozmadan 4 eklemek için 2'nin artık 4'ü göstermesi gerekir.
- Bazen liste üzerinde tersten gitmek de gerekebilir.
- Bu ihtiyaç sürekli ise her düğüme önceki elemanı gösteren bir gösterici eklenir:
  - `prev`: önceki düğümün adresi (previous pointer)
  - `next`: sonraki düğümün adresi (next pointer)
- Yapıda `struct node` tipinde bir `next`'e ek olarak `struct node` tipinde bir `prev` alanı bulunur.
- İlk düğümün `prev`'i her zaman NULL'dur, çünkü ondan önce eleman yoktur. Son düğümün `next`'i de NULL'dur.
- İstenirse çift yönlü liste de dairesel yapılabilir. Hoca bu derste bunu anlatmadı.

Başa ekleme (ekrandan, 01:35:05):

```c
void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    if (head == NULL)
    {
        head = newNode;
        /* head->prev = NULL;  create içinde yapıldığı için gerek yok */
    }
    else
    {
        head->prev = newNode;
        newNode->next = head;
        head = newNode;
    }
}

NODE *create()
{
    /* ... yer ayır (allocate) ... */
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}
```

- Örnek: liste 5 (1000) ⇄ 7 (1200). 13 (1500) başa eklenir.
  - `head->prev = newNode`: 5'in `prev`'i 1500 olur.
  - `newNode->next = head`: 13, 1000'i gösterir.
  - `head = newNode`: `head` 1500 olur.
  - 13'ün `prev`'i `create` içinde NULL yapılmıştır.
- Hoca önce boş liste durumunda `head->prev = NULL` yazdı. Bir öğrenci neden yalnızca `prev`'in NULL yapıldığını sorunca bu satırı sildi ve `create` içine `newNode->prev = NULL` ekledi.
- Sona ve araya ekleme öğrencilere bırakıldı.

Çift yönlü listenin değerlendirmesi:

	Açıklama
Avantaj	Önceki elemanın adresini geçici bir `prev` değişkeninde tutmak gerekmez. Her an her düğümün öncekine erişilir.
Dezavantaj	Tek yönlü listede silme ve eklemede tek bir geçici gösterici yeterliydi. Burada her düğüm için ayrı bir gösterici tutulur. Bu yer kaybıdır.

Ders sonrası notlar

- **`printList` döngü koşulu:** tahtadaki `for (p = last->next; p != last->next; p = p->next)` döngüsü hiç çalışmaz. `p` ilk adımda zaten `last->next`'e eşittir, bu yüzden koşul baştan yanlıştır ve hiçbir şey yazdırılmaz. İlk elemanı yazdırıp sonra kontrol eden bir `do-while` kullanılmalıdır. Aşağıdaki kod derste yazılmadı. Ajan tarafından `gcc -Wall` ile derlenip küçük bir listeyle (9, 7, 4, 8) test edildi. Tahtadaki hâli hiçbir şey yazdırmadı, düzeltilmiş hâli 9, 7, 4, 8 yazdırdı.

```c
void printList()
{
    NODE *p;
    if (last == NULL)
    {
        printf("List is empty\n");
        return;
```

```
    }
    p = last->next;              /* ilk eleman */
    do
    {
        printf("%d\n", p->value);
        p = p->next;
    } while (p != last->next);   /* tekrar ilk elemana gelince dur */
}
```

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.